Práctica 1 · Usuarios, perfiles y módulos

> **RA3 · CE a, b, c**
>
> **Objetivo:** Preparar la rama de RA3 con WP-CLI y los registros de nginx, crear usuarios con distintos perfiles y comprobar qué puede hacer cada uno, y elegir, instalar y configurar los plugins que piden los requisitos del sitio.

Cada paso tiene las órdenes y tres cuadros: **¿Qué hace esto?** (léelo antes de ejecutar), **Qué debes ver** (si no lo ves, no avances) y **Ojo** (errores típicos). Ve completando `INFORME_RA3.md` en la raíz del repositorio.

Dónde trabajar: aula o Codespaces

Puedes hacer el bloque en el equipo del aula o en GitHub Codespaces. **Elige uno y quédate en él todo el bloque**: los usuarios, plugins y contenidos viven en los volúmenes de esa máquina.

	Equipo del aula	GitHub Codespaces
Dirección del sitio	`http://localhost:8080` (otros equipos: `http://TU-IP:8080`)	La de la pestaña PORTS: `https://…-8080.app.github.dev`
Probar con otro usuario	Ventana privada del navegador	Ventana privada con el puerto en **Public**
Copias automáticas (práctica 2)	Con `cron`	No: la máquina se para. Copia a mano al acabar
Al terminar la clase	`git push`	`git push` y **parar** el codespace

Paso 1 · Crear la rama de RA3

```
git switch ra2-cms        # o main, si ya fusionaste RA2
git pull
git switch -c ra3-admin
mkdir -p logs datos backups scripts
```

> **¿Qué hace esto?**
>
> Todo el bloque se trabaja en la rama `ra3-admin`, que parte del final de RA2. Las cuatro carpetas se crean ahora, con tu usuario, para que Docker no las cree después como `root`.

> **Qué debes ver**
>
> `Switched to a new branch 'ra3-admin'` y, con `ls`, las carpetas nuevas junto a `docker-compose.yml`.

> **Ojo**
>
> Si `git switch ra2-cms` dice que tienes cambios sin guardar, haz commit antes (o pregunta).

Paso 2 · Ampliar el `docker-compose.yml`

Añade dos cosas: una línea en el servicio `nginx` y un servicio nuevo, `wpcli`, al final de `services:` (antes de `volumes:`).

```
  nginx:
    ...
    volumes:
      - wp_data:/var/www/html
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./logs:/var/log/nginx          # NUEVO

  wpcli:                               # NUEVO SERVICIO
    image: wordpress:cli-php8.3
    profiles: ["herramientas"]
    user: "33:33"
    environment:
      HOME: /tmp
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_NAME: ${WP_DB_NAME}
      WORDPRESS_DB_USER: ${WP_DB_USER}
      WORDPRESS_DB_PASSWORD: ${WP_DB_PASSWORD}
    volumes:
      - wp_data:/var/www/html
      - ./datos:/datos:ro
    depends_on:
      - db
```

Línea	Qué hace
`./logs:/var/log/nginx`	nginx escribe sus registros (`access.log`, `error.log`) en la carpeta `logs` del proyecto. Los analizaremos en la práctica 5.
`image: wordpress:cli-php8.3`	Imagen oficial de WordPress con **WP-CLI**, la herramienta de línea de órdenes de WordPress.
`profiles: ["herramientas"]`	`docker compose up -d` **no** arranca este servicio. Solo se usa cuando lo pedimos con `docker compose run`.
`user: "33:33"`	Se ejecuta como `www-data` (UID 33), el mismo usuario que PHP-FPM. Así los ficheros que cree tendrán el propietario correcto.
`HOME: /tmp`	WP-CLI guarda su caché en la carpeta personal; `/tmp` siempre se puede escribir.
`wp_data:/var/www/html`	Ve los mismos ficheros de WordPress que `nginx` y `wordpress`.
`./datos:/datos:ro`	Carpeta para pasarle ficheros a WP-CLI (los usaremos en la práctica 3). Solo lectura.

```
docker compose config -q
docker compose up -d
```

Qué debes ver

`config -q` no escribe nada si el YAML es correcto.

`up -d` muestra `Container …-nginx-1 Recreated` y `Started`: nginx se recrea porque ha cambiado su configuración. `wordpress` y `db` siguen igual (`Running`).

Ojo

Desde ahora `docker compose logs nginx` ya **no** muestra las visitas: van al fichero `logs/access.log`. Para verlas en directo: `tail -f logs/access.log`.

Paso 3 · El atajo `./wp` y el `.gitignore`

Escribir `docker compose run --rm -T wpcli wp …` cada vez es largo. Crea en la raíz del repositorio un fichero llamado `wp`:

```
cat > wp << 'EOF'
#!/usr/bin/env bash
# Atajo para usar WP-CLI con el stack del proyecto
exec docker compose run --rm -T wpcli wp "$@"
EOF
chmod +x wp

cat >> .gitignore << 'EOF'
backups/
exportaciones/
logs/
informes/
EOF

./wp core version
./wp user list
```

¿Qué hace esto?

`run --rm` crea un contenedor de `wpcli`, ejecuta la orden y lo borra. `-T` evita la terminal virtual para poder guardar la salida en un fichero sin estropearla. `"$@"` pasa todo lo que escribas detrás de `./wp`.

En `.gitignore` añadimos las carpetas que **nunca** deben subir a GitHub: copias, exportaciones, registros e informes.

Qué debes ver

La primera vez se descarga la imagen `wordpress:cli-php8.3`.

`core version` muestra `7.1` (o `7.1.x`).

`user list` muestra una tabla con tu usuario administrador.

Ojo

`Permission denied` al ejecutar `./wp`: falta `chmod +x wp`.

`Error establishing a database connection`: tu `.env` no tiene las variables `WP_DB_NAME`, `WP_DB_USER` y `WP_DB_PASSWORD` (las de RA2). Revísalo con el profesorado.

Paso 4 · Roles y usuarios

WordPress no da permisos a cada persona uno a uno: asigna un **rol** a cada usuario, y cada rol es un conjunto de **capacidades** (`publish_posts`, `edit_others_posts`, `manage_options`…). Vamos a montar el equipo de la web del ciclo. Crea el primero desde el panel: **Usuarios → Añadir nuevo**: usuario `coord.web`, correo `coord.web@example.com`, rol **Editor**. Los demás, con WP-CLI:

Rol	Puede	No puede
Administrador	Todo: ajustes, plugins, temas, usuarios	—
Editor	Publicar y editar entradas y páginas de **cualquiera**, moderar comentarios, gestionar categorías	Tocar ajustes, plugins, temas o usuarios
Autor	Escribir, publicar y borrar **sus** entradas; subir imágenes	Editar lo de otros, crear páginas
Colaborador	Escribir **sus** entradas y enviarlas a revisión	Publicar ni subir imágenes
Suscriptor	Leer y editar su perfil	Escribir contenido

```
./wp role list
./wp cap list contributor

./wp user create profe.redes profe.redes@example.com --role=author --display_name="Profe de Redes"
./wp user create alumna.fct alumna.fct@example.com --role=contributor --display_name="Alumna en prácticas"
./wp user create familia.garcia familia.garcia@example.com --role=subscriber
./wp user list --fields=ID,user_login,roles
```

> **¿Qué hace esto?**
> `user create` necesita usuario y correo. Si no le das contraseña, genera una segura y la muestra **una sola vez**.

> **Qué debes ver**
> `cap list contributor` muestra solo 3 capacidades (`read`, `edit_posts`, `delete_posts`): no tiene `publish_posts` ni `upload_files`.
>
> Cada orden termina con `Success: Created user N.` y una línea `Password: …`. **Apúntala**: la necesitas en el paso siguiente.
>
> La lista final tiene 5 usuarios con 5 roles distintos.

> **Ojo**
> Los correos `@example.com` son un dominio reservado para ejemplos: no llegan a nadie, y eso es lo que queremos en clase.
>
> Si pierdes una contraseña: `./wp user update alumna.fct --user_pass='NuevaClave.2026'`.
>
> Anota en `INFORME_RA3.md` qué capacidad permite publicar y cuál subir ficheros.

Paso 5 · Comprobar qué puede hacer cada perfil

Abre una **ventana privada** del navegador (`Ctrl+Mayús+N`) y entra en `/wp-admin` con cada usuario. Mantén tu sesión de administrador en la ventana normal.

1. **alumna.fct (Colaboradora):** crea una entrada «Mi primera semana en prácticas». El botón dice **Enviar para revisión**, no Publicar. Intenta añadir una imagen: no puede subirla.
2. **coord.web (Editora):** en **Entradas → Pendientes** aparece la entrada de la alumna. Revísala y publícala. Mira el menú: no hay **Plugins** ni **Ajustes**.
3. **profe.redes (Autor):** publica una entrada propia. Intenta editar la de la alumna: no puede.
4. **familia.garcia (Suscriptora):** solo ve su perfil.

> **Qué debes ver**
> Cada perfil ve un menú lateral distinto. Rellena la tabla de la sección 1 de `INFORME_RA3.md` con lo que puede y no puede hacer cada uno.

> **Ojo**
> **En Codespaces:** si el puerto 8080 está en **Private**, la ventana privada te pedirá iniciar sesión en GitHub. Durante esta sesión ponlo en **Public** (pestaña PORTS → clic derecho → Port Visibility).

Paso 6 · Un rol a medida y el registro cerrado

Necesitamos a alguien que revise y publique lo que escriben otros, pero que **no pueda borrar** nada. Ningún rol encaja, así que creamos uno a partir de Editor y le quitamos capacidades:

```
./wp role create revisor "Revisor" --clone=editor
./wp cap remove revisor delete_posts delete_others_posts delete_published_posts \
    delete_private_posts delete_pages delete_others_pages delete_published_pages \
    delete_private_pages manage_categories
./wp user create revisor.fp revisor.fp@example.com --role=revisor
./wp cap list revisor | grep delete || echo "Sin capacidades de borrado"

./wp option get users_can_register     # debe dar 0
./wp option get default_role           # debe dar subscriber
```

¿Qué hace esto?

WordPress **no tiene grupos de usuarios**: el rol es el grupo. Crear un rol nuevo es la forma de tener un perfil a medida. Los plugins de gestión de roles hacen lo mismo con una interfaz gráfica.

Si el registro abierto estuviera activado con rol Administrador, cualquiera podría crearse una cuenta con control total: es uno de los fallos de configuración más graves.

Qué debes ver

`Success: Role with key 'revisor' created. Cloned capabilities from 'editor'.` y después `Success: Removed 9 capabilities…`.

La última orden muestra `Sin capacidades de borrado`.

`0` y `subscriber`: nadie puede registrarse solo y, si se activara, entraría como Suscriptor.

Ojo

Entra como `revisor.fp`: puede publicar la entrada pendiente de otro, pero no aparece el enlace **Papelera**.

Paso 7 · Qué es un módulo y cómo elegirlo

La web del ciclo tiene que cumplir estos requisitos. Para cada uno compara al menos **dos** plugins y anota la elección en la sección 2 de `INFORME_RA3.md`.

Requisito	Sugerencia de partida
R1. Formulario de contacto que **guarde** los mensajes	Contact Form 7 + Flamingo
R2. Aviso de cookies (LSSI y RGPD)	Busca y compara dos: `./wp plugin search cookie`
R3. Registro de actividad de los usuarios	Simple History

```
./wp plugin list
docker compose exec wordpress ls wp-content/plugins

./wp plugin search cookie --fields=name,slug,active_installs,rating,last_updated --per-page=6
```

¿Qué hace esto?

Un **plugin** (módulo) es código PHP que se engancha a WordPress para añadir funciones sin tocar el núcleo. Cada uno es una carpeta dentro de `wp-content/plugins`, en el volumen `wp_data`.

Qué debes ver

`plugin list` muestra nombre, estado (`active`/`inactive`), versión y si hay actualización. Normalmente vienen `akismet` y `hello` (Hello Dolly) sin activar.

Una tabla con instalaciones activas, valoración (sobre 100) y fecha de la última actualización.

Ojo

Criterios: **instalaciones activas**, **última actualización** (desconfía de más de un año), **compatible hasta** la versión 7.1, **valoraciones** y foro de soporte con respuestas. Un plugin abandonado es un riesgo de seguridad.

Paso 8 · R1: formulario de contacto que guarda los mensajes

1. En el panel aparece el menú **Contacto**. Edita el «Formulario de contacto 1»: añade un campo desplegable `ciclo` con las opciones `ASIR | DAW | SMR`.
2. En la pestaña **Configuración adicional** escribe `skip_mail: on` y guarda.
3. Edita la página **Contacto** y añade el bloque **Contact Form 7** con ese formulario. Publica.
4. Abre la página como visitante y envía un mensaje. Después, en **Flamingo → Mensajes entrantes**, debe aparecer.

```
./wp plugin install contact-form-7 flamingo --activate
./wp post create --post_type=page --post_title="Contacto" --post_status=publish
```

> **¿Qué hace esto?**
> Integrar un módulo es más que activarlo: hay que configurarlo y colocarlo donde lo necesita el sitio. `skip_mail: on` le dice a Contact Form 7 que no intente enviar correo; Flamingo guarda cada envío en la base de datos.

> **Qué debes ver**
> Mensaje verde «Gracias por tu mensaje. Ha sido enviado.» y el mensaje en Flamingo con el campo `ciclo`.

> **Ojo**
> Sin `skip_mail: on` verás un error naranja: el contenedor no tiene servidor de correo. No es un fallo del plugin.

Paso 9 · R2 y R3: aviso de cookies y registro de actividad

1. Instala el plugin elegido **desde el panel**: **Plugins → Añadir nuevo plugin**, búscalo, **Instalar ahora** y **Activar**.
2. Recorre su asistente: idioma español, texto del aviso, botones **Aceptar** y **Rechazar** con el mismo peso visual, y enlace a una página de política de cookies (créala vacía si no existe).
3. Abre el sitio en una ventana privada: el aviso aparece; tras aceptarlo no vuelve a salir.
4. Instala el registro de actividad desde la terminal: `./wp plugin install simple-history --activate`. En el Escritorio aparece **Historial**.

¿Qué hace esto?

Has instalado un módulo de las dos formas: con WP-CLI (rápido, se puede automatizar) y con el panel (lo que hará un cliente). El resultado es el mismo: una carpeta nueva en `wp-content/plugins` y una fila en `active_plugins`.

Simple History registra quién entra, quién falla al entrar, qué se publica y qué plugins se activan. Lo instalamos **ya** para que vaya acumulando datos hasta la práctica 5, donde haremos el informe de accesos.

Qué debes ver

`./wp plugin list` muestra el plugin nuevo como `active`.

Ojo

Si el panel pide datos FTP para instalar, los permisos de `wp-content` no son de `www-data`. Avisa: se arregla con `docker compose exec wordpress chown -R www-data:www-data wp-content`.

Entra una vez como `alumna.fct` con una contraseña **mal** escrita y luego bien: Simple History registra el intento fallido.

Paso 10 · Eliminar lo que sobra y guardar el trabajo

```
./wp plugin deactivate akismet
./wp plugin list --status=inactive
./wp plugin delete akismet hello
./wp plugin list

git add docker-compose.yml wp .gitignore INFORME_RA3.md
git commit -m "RA3: WP-CLI, usuarios y módulos"
git push -u origin ra3-admin
```

¿Qué hace esto?

Desactivar deja de ejecutar el plugin, pero su código sigue en el disco y sus datos en la base de datos. **Eliminar** borra la carpeta (y, si el plugin lo prevé, sus datos). Un plugin instalado pero inactivo puede seguir siendo explotable si tiene una vulnerabilidad.

Los usuarios y los plugins **no** van a GitHub: viven en los volúmenes. Por eso la práctica 2 es de copias de seguridad.

Qué debes ver

Tras `delete`, `Success: Deleted 2 of 2 plugins.` y la lista ya no los muestra.

En GitHub aparece la rama `ra3-admin`, sin la carpeta `logs/`.

Ojo

No elimines Contact Form 7, Flamingo, Simple History ni el de cookies: los necesitamos.

Completa las secciones 1 y 2 de `INFORME_RA3.md` antes del commit.

Si algo falla

Síntoma	Solución
`./wp: Permission denied`	Falta `chmod +x wp`.
`no such service: wpcli`	El servicio no está en el compose o la sangría es incorrecta. `docker compose config -q`.
`Error establishing a database connection` al usar `./wp`	Faltan `WP_DB_NAME`, `WP_DB_USER` o `WP_DB_PASSWORD` en el `.env`.
Avisos sobre `/tmp/.wp-cli/cache`	Inofensivos: WP-CLI no puede usar su caché. No afectan al resultado.
La carpeta `logs` (o `datos`) es de root	La creó Docker porque no existía. Sin `sudo`: `docker compose down`, `docker run --rm -v "$PWD":/w alpine rm -rf /w/logs`, `mkdir logs`, `docker compose up -d`.
El panel pide datos de FTP al instalar un plugin	`wp-content` no es de `www-data`: `docker compose exec wordpress chown -R www-data:www-data wp-content`.